

9

FILE SYSTEMS

9.1 File

- Collection of logically related entities/records

9.1.1 Attributes

- File Name
- File Type
- File Size
- Location of file
- Generation date
- Last modified date
- Permission
- Owner/Author
- Password

9.2 File Context

File context is stored in file control block (FCB)

File will have various types–

- .doc
- .txt
- .pdf
- .exe
- .obj
- .dll
- .png
- .apk
- .xls/.xlsx
- .jpg
- .mp3
- .mp4
- .avi/.flv/.mkv/.3gp
- .c/.cpp/.java/.xml/.html

9.3 Operations performed on file

- create
- open
- write
- read
- hide
- save
- save as
- close
- copy
- paste
- cut
- delete
- rename
- send/share
- print

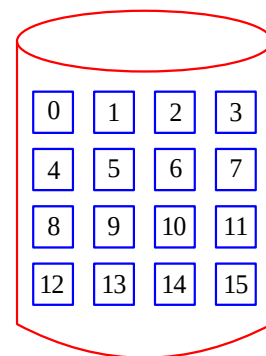
9.4 Access methods

- Sequential Access
- Random Access
- For better classification of files, files will be stored in directory.

9.5 Disk Space Allocation Methods

9.5.1 Contiguous Allocation

File	Starting Disk Block Address	Size w.r.t no. of DB
Abc.docx	2	4
Xyz.docx	9	5



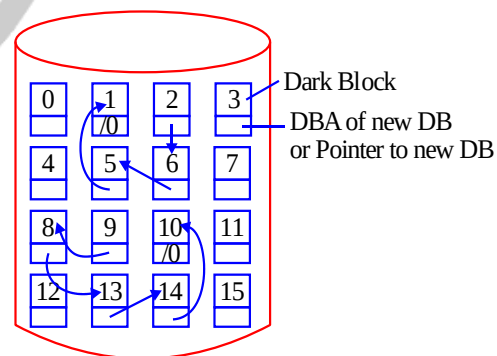
- In contiguous allocation, whenever file is created, disk blocks are allocated in a continuous manner.
- Every file is associated with two parameters
 - (1) Starting DBA
 - (2) Size
- Increasing the file size may not be possible always.
- It suffers from external fragmentation.
- Internal fragmentation may exist in the last disk block of the file.
- It supports both sequential and random access of file.

Starting disk block Address + Offset Value $\Rightarrow$ Random location

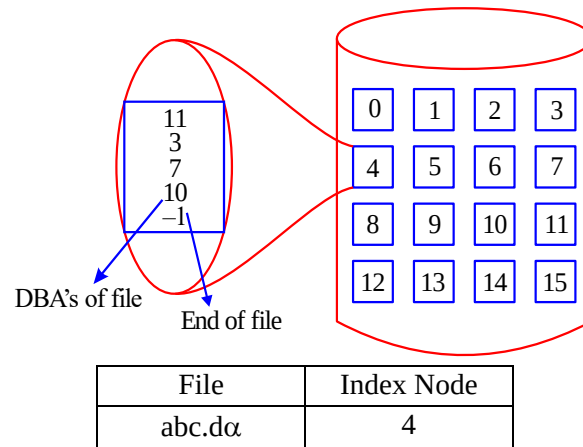
9.5.2 Linked Allocation/Non-contiguous allocation

File	Starting DBA	Ending DBA
abc.docx	2	1
xyz.docx	9	10

- In the linked allocation disk blocks are allocated in a non-continuous manner.
- Every file is associated with 2 parameters–
 - (1) Starting DBA
 - (2) Ending DBA
- Increasing file size is always possible if free disk block is available.
- There is no external fragmentation.
- Internal fragmentation may exist in last disk block of the file.
- There is an overhead of maintaining pointer in every disk blocks.
- If the pointer of any disk block is lost, the file will be truncated.
- It supports only sequential access of the file.



9.5.3 Indexed Allocation



- In the indexed allocation, every file is associated with its own index node.
- Index node contains all the disk block address of the file.
- There is no external fragmentation but internal fragmentation may exist in the last disk block of file.
- If the file is very large, then one disk block may not be sufficient to store all the disk block addresses of the file.
- If the file is very small, then it is waste of using one entire disk blocks. Just to store, the addresses.

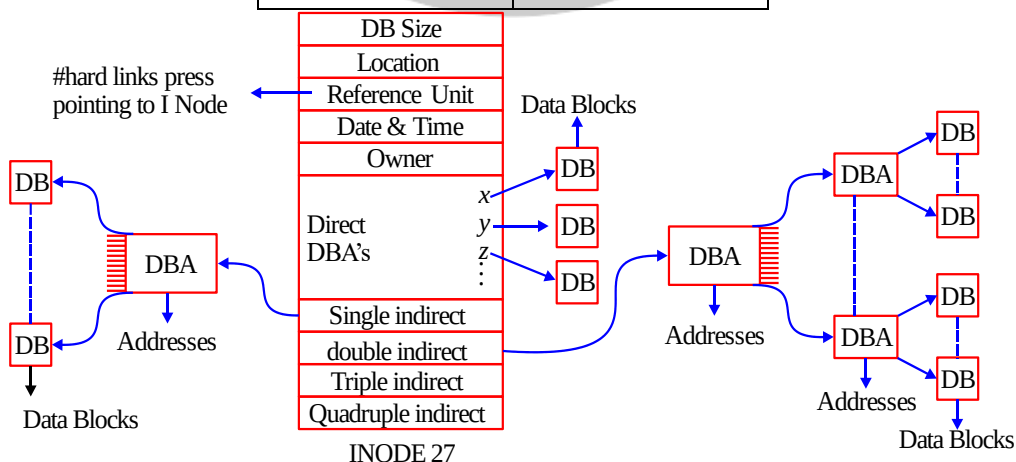
9.7 UNIX I-NODE IMPLEMENTATION

9.7.1 An extension of indexed allocation

$$\text{Total size of file system} = \left[\# \text{Direct DBA's} + \left(\frac{\text{DB Size}}{\text{DBA}} \right) + \left(\frac{\text{DB Size}}{\text{DBA}} \right)^2 + \left(\frac{\text{DB Size}}{\text{DBA}} \right)^3 + \dots \right] \times \text{DB Size}$$

#Single indirect DBA #Double Index DBA #Triple Index DBA

File	I-NODE
abc.docx	27



9.8 DISK FREE SPACE MANAGEMENT

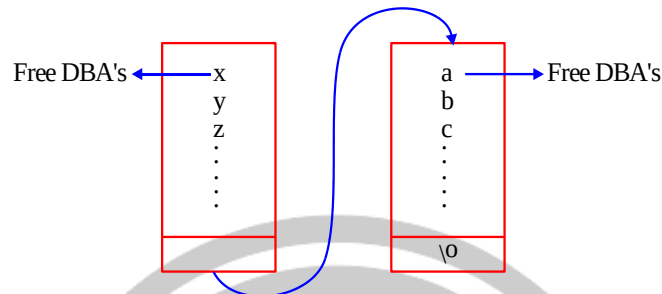
Size of Disk = 20 MB

DB Size = 1 KB

DBA = 16 Bits = 2 B

$$\# \text{ Disk blocks available on given disk} = \frac{\text{Size of disk}}{\text{DB size}} = \frac{20 \times 2^{20} B}{2^{10} B} = 20 K$$

9.8.1 Free List Approach



- In the free list approach, some disk blocks are used just to store the free disk block addresses.

$$\# \text{ Disk block addresses possible to store in one disk block} = \frac{\text{DB Size}}{\text{DBA}} = \frac{2^{10}}{2} = 2^9$$

2^9 addresses $\rightarrow$ 1 disk block

$$20 K \text{ addresses} \rightarrow \frac{1}{2^9} \times 2^{11} \times 10 \rightarrow 2^2 \times 10 = 40 \text{ disk block}$$

9.8.2 Bit Map Approach

- In Bit map approach, every disk block were be mapped with 1 binary bit

Free Disk Blocks

0, 2, 4, 6, 9, 11,

Busy Disk Blocks

1, 3, 5, 7, 8, 10,

Disk blocks we can map in 1 disk block = 1 K $\times$ 8 bits = 8 K bits.

8 K bits $\rightarrow$ 1

$$20 K \text{ bits} \rightarrow \frac{1}{8K} \times 20K = 2.4 = 3$$

